ZG536 LLMS FOR GENAI

Foundations of Large Language Models

From a plain sentence to a predicted next word: the five-stage pipeline inside every LLM

BUILT FROM: THE SESSION 1 HANDOUT, SLIDES AND RECORDING · EVERY EXAMPLE WORKED OUT IN FULL

How to use this companion. This companion explains Session 1 from scratch, in plain English. Every idea starts with an everyday picture. Then the mechanism is built up one step at a time. Every example is worked out in full, with real numbers. **THE INTUITION** states the core idea in one breath. **EVERYDAY PICTURE** ties it to real life before any mechanism. **WORKED EXAMPLE**, on a grey background, walks a real calculation through with no skipped steps. **WATCH OUT** flags the common mistake. **WAIT — REALLY?** holds a result that sounds wrong but is true. **KEY TAKEAWAY** is the one sentence to remember. Read it in order; each section leans on the one before.

1 Try it first: trace one sentence through an LLM

Before any theory, follow one short sentence as far as you can, by feel, and see exactly where the trail runs out.

Take the sentence *"The dog loved the movie"* and imagine handing it to a large language model. Underneath everything, that model is just a very large pile of numbers and arithmetic. Three honest questions:

> *How does a model built only from numbers "read" the word dog? What does it mean for the model to "remember" that dog came before movie, not after? And once it has read the whole sentence, how does it pick the next word?*

You cannot answer any of these yet, and that is fine — by the end of this session you can. Every LLM, from the smallest to the largest, pushes raw text through the same five-stage pipeline, once per word it generates.

Section 2 explains what "predicting the next word" even means, as a probability. Section 3 turns text into token IDs. Section 4 turns IDs into meaningful, ordered vectors. Section 5 turns those vectors into an actual predicted word. Nothing here needs attention or Transformers in detail yet. Those are reviewed, not retaught, starting Session 3 — if they are not solid yet, work through the Prerequisites Primer first.

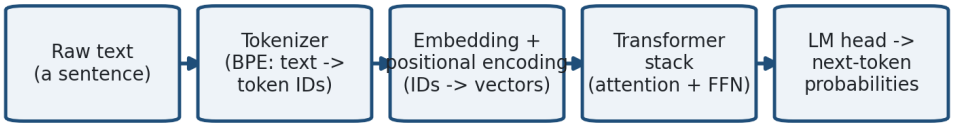


Figure 1. The five-stage pipeline every sentence travels through. Sections 2 to 5 build each stage, one at a time.

2 Probabilistic language models: predicting the next word, on purpose

A language model is, formally, nothing more than a rule for guessing the next word from the words said so far.

Formally, a language model assigns a probability to what word comes next, given everything written before it:

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

THE INTUITION

This is the whole job, stated precisely. Everything else in this course — tokenization, embeddings, attention, the Transformer — exists only to compute this one probability well. It runs over and over, one word at a time.

EVERYDAY PICTURE

If you have watched every film in a franchise, in order, you can guess what kind of scene follows a big battle. You do this with decent confidence, not certainty: usually it is a quiet aftermath, not another battle immediately after. You did not memorise a rule. You absorbed the pattern from dozens of examples. A language model does the same thing with text. Having "read" huge amounts of it, it estimates how likely each possible next word is, given everything written so far.

WORKED EXAMPLE · *a tiny count-based language model, by hand*

Suppose your entire training corpus is just four short sentences. They are: "the dog barked at the mailman" and "the dog barked at the cat". Also: "the dog barked at the mailman" again, and "the dog slept on the mat".

Step 1 **Find every exact match for the 5-word context.** The phrase "the dog barked at the" occurs 3 times in total, across the corpus.

Step 2 **Count what follows each time.** It is followed by "mailman" twice and by "cat" once — 3 continuations observed.

Step 3 **Turn counts into probabilities.** So $P(\text{mailman}) = 2/3 \approx 0.67$ and $P(\text{cat}) = 1/3 \approx 0.33$.

Step 4 **Predict.** This tiny model predicts "mailman". It is the single most likely next word, purely because it appeared more often in the small corpus it was shown.

This is literally what an **n-gram model** does. It counts how often each word follows the previous fixed number of words, directly from a frequency table. No understanding, no meaning — only counting.

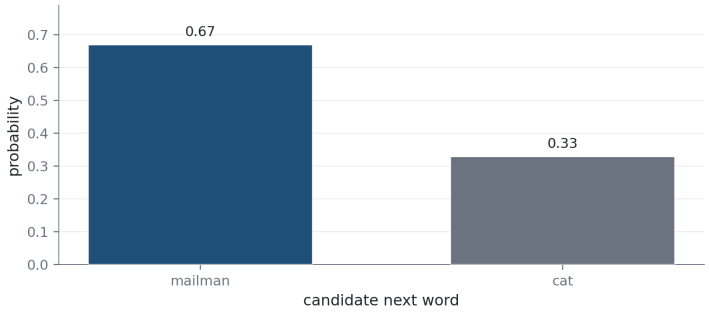


Figure 2. The tiny model's full output: a probability for every word it has ever seen follow this exact context.

Approach	How it estimates the probability	Where it breaks
N-gram models	Count how often a word follows the previous $N-1$ words, directly from frequency tables	Only ever "sees" the last $N-1$ words; no sense of meaning
Neural language models	A small neural network learns a continuous representation of context instead of raw counts	Still a fixed-size context window; learns slowly from scratch
Transformer-based LLMs	Attention lets the model look arbitrarily far back, weighting every earlier word by relevance	The hard problems move to cost, data, and reliability (later sessions)

Three ways to estimate the same probability, in increasing order of capability and cost.

WATCH OUT

An n-gram model's blind spot is structural, not a bug you can tune away. If $N = 4$, it is mathematically incapable of noticing a word five positions back, no matter how relevant that word is. This is the exact same fixed-window memory problem that limits a plain RNN, covered in the Prerequisites Primer.

KEY TAKEAWAY

A language model's whole job is estimating $P(w_t \mid w_1, \ldots, w_{t-1})$. N-grams estimate it by counting. Neural and Transformer models estimate it by learning a representation — the rest of

> *this session builds that representation, piece by piece.*

3 Tokenization and BPE: text into reusable pieces

A sentence must become a short list of known pieces before any arithmetic can run on it — not whole words.

A model cannot store a separate meaning for every possible word. New words, typos, and names appear constantly. A fixed whole-word vocabulary would either explode in size, or fail on anything it has not seen before. **Byte Pair Encoding (BPE)** solves this by building a vocabulary of reusable *sub-word* pieces instead of whole words.

THE INTUITION

Think of whole-word vocabularies as pre-built plastic statues: one fixed piece per word, nothing reusable, and no way to build something new. BPE pieces are LEGO bricks. They are a modest set of small, reusable pieces that combine to build almost any word, including ones never seen before.

WORKED EXAMPLE · *BPE on the toy corpus {low×5, lower×2, lowest×2, newer×6, wider×3}*

Step 1 **Start from characters.** Every word is split into characters, each ending with an end-of-word marker `_`. The five words become `l o w _` (×5) and `l o w e r _` (×2). The next two are `l o w e s t _` (×2) and `n e w e r _` (×6). The last is `w i d e r _` (×3).

Step 2 **Count every adjacent pair across the whole weighted corpus.** The pair `(e, r)` appears in "lower" (×2), "newer" (×6), and "wider" (×3) = 11 times — the most frequent pair. Merge it: `e r` → `er`.

Step 3 **Recount.** Now `(er, _)` appears $2+6+3=11$ times (every word that just got "er" immediately ends in it). Merge it: `er` + `_` → `er_`.

Step 4 **Recount again.** `(n, e)` appears in "newer" ×6. But `(l, o)` appears in "low" (×5), "lower" (×2), and "lowest" (×2) — 9 times total, now the winner. Merge `l o` → `lo`.

The process continues for a fixed number of merges, or until a target vocabulary size is reached. Notice what falls out for free. "Lower" and "newer" end up sharing the learned piece `er_`, and "low", "lower", "lowest" end up sharing `lo`. The algorithm discovers word-forming patterns purely from frequency — nobody hand-coded that "*-er*" is a comparative suffix.

EVERYDAY PICTURE

A rare dog breed name like "Labradoodle" is almost never seen whole in training text. BPE has very likely already learned "Labra" (from Labrador) and "doodle" (from

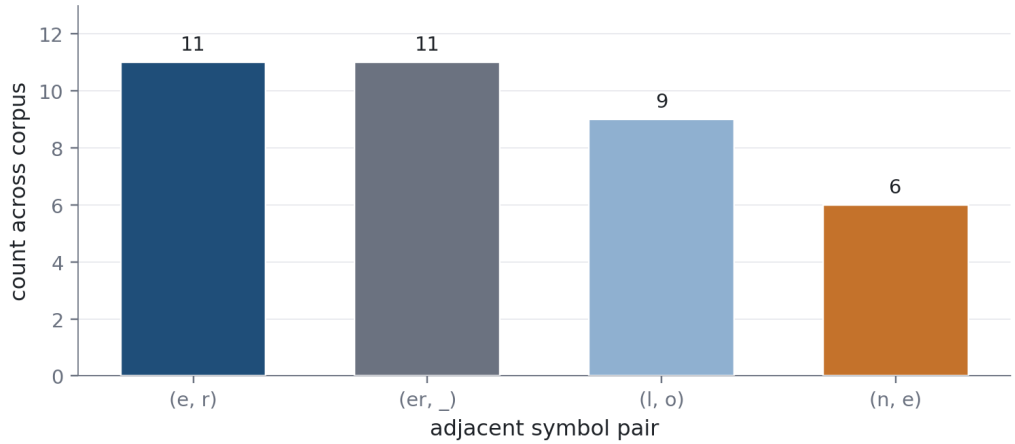


Figure 3. The first merge decision: BPE always merges whichever adjacent pair is currently most frequent across the whole corpus.

> Poodle-cross breed names) as separate, common pieces from other words. The rare whole word is rebuilt from pieces the model already knows well. The same trick handles typos, brand names, and words in languages the model has seen very little of.

Byte-level BPE (what GPT-style tokenizers actually use) runs this exact merge process starting from raw UTF-8 *bytes* rather than Unicode characters. With only 256 possible starting symbols, every byte value, there is no such thing as an unrepresentable string. Any text in any language or emoji can always be tokenized, possibly just as more, smaller pieces for rare scripts.

> **WAIT — REALLY?**
>
> Tokenization is also why LLMs can be strangely bad at some tasks that look trivial to a human. Counting letters inside a word is a good example. The model never sees individual characters directly. It only sees whichever sub-word pieces BPE happened to merge that word into. A letter-counting question is secretly a question about those pieces, not about reasoning ability.

> **KEY TAKEAWAY**
>
> *BPE builds a vocabulary of reusable sub-word pieces by repeatedly merging the most frequent adjacent pair in the training corpus. It handles rare and unseen words gracefully, at the cost of the model never directly seeing individual letters.*

4 Embeddings and positional encoding: meaning, then order

Each token gets a vector that captures its meaning, plus a second vector that records where it sits in the sentence.

A token ID on its own is just an arbitrary number — it carries no sense of meaning. The next stage looks that ID up in a learned **embedding table**. That is a big lookup table that returns a vector of numbers meant to capture what the token *means*. That meaning is learned entirely from how the token is used across huge amounts of text.

THE INTUITION

Two tokens that tend to appear in similar contexts end up with similar vectors. Nobody labelled them as similar. The model is trained to predict text well, and giving similar words similar vectors makes that prediction easier.

EVERYDAY PICTURE

This is exactly how a streaming service recommends movies. Two films watched by the same kind of viewer get pulled close together in the service's internal "taste space". This happens even in similar contexts, with nobody tagging them. A word embedding space works the same way, just for words instead of films.

WORKED EXAMPLE · *comparing three toy word vectors by hand*

Suppose a (deliberately tiny, two-number) embedding table gives: dog $= (0.9, 0.1)$, cat $= (0.8, 0.2)$, car $= (0.1, 0.9)$.

Step 1 **Compute the dot product of dog and cat.** $0.9 \times 0.8 + 0.1 \times 0.2 = 0.72 + 0.02 = 0.74$.

Step 2 **Compute each vector's length.** $|\text{dog}| = \sqrt{0.9^2 + 0.1^2} = \sqrt{0.82} \approx 0.91$, and $|\text{cat}| = \sqrt{0.8^2 + 0.2^2} = \sqrt{0.68} \approx 0.82$.

Step 3 **Divide to get cosine similarity.** $0.74/(0.91 \times 0.82) \approx 0.74/0.75 \approx 0.99$ — very close to 1, meaning "dog" and "cat" point in almost the same direction.

Step 4 **Repeat for dog and car.** Dot product $= 0.9 \times 0.1 + 0.1 \times 0.9 = 0.18$; $|\text{car}| \approx 0.91$; cosine similarity $\approx 0.18/0.83 \approx 0.22$ — far less similar.

"Dog" sits close to "cat" and far from "car" in this toy space. This holds purely as numbers — no dictionary definition was involved anywhere in the calculation.

A separate, lightweight **embedding model** does exactly this lookup as its entire job. It is used directly for search and retrieval, not buried inside a generative LLM. That matters in Session 12's retrieval pipeline, where running a full LLM just to get a vector would be wasteful.

Meaning alone is not enough, though. "The dog bit the man" and "the man bit the dog" use the exact same words and the exact same meaning vectors. Yet they mean opposite things. A **positional encoding** vector, one per position in the sentence, is added on top of each token's

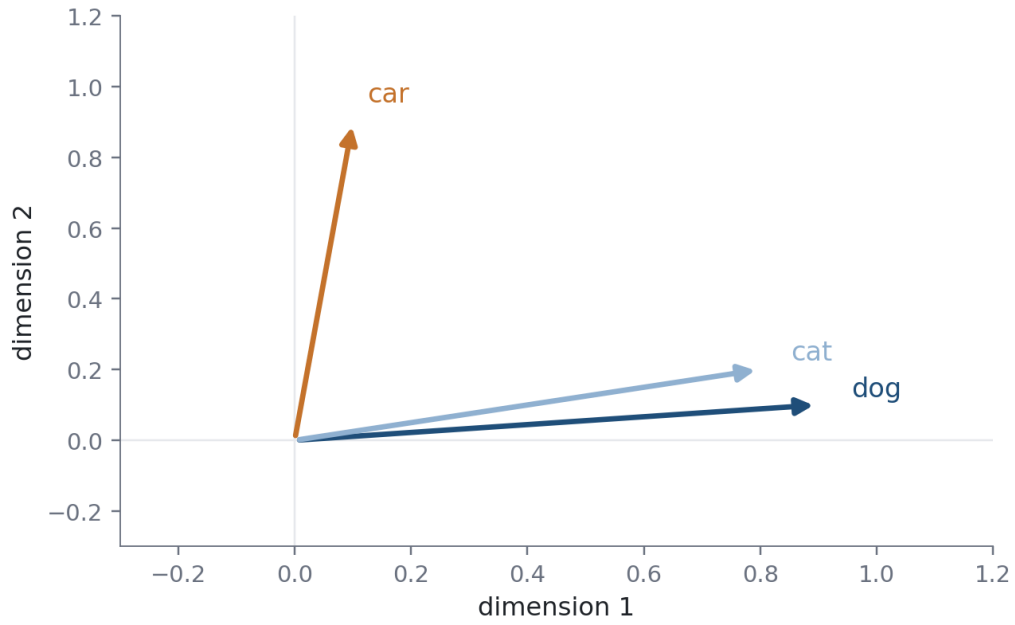


Figure 4. The toy embedding space from the worked example: "dog" and "cat" point in nearly the same direction; "car" points elsewhere.

meaning vector. The two are literally summed, element by element, before anything enters the Transformer stack.

> **EVERYDAY PICTURE**
>
> A positional encoding is like the timestamp on a subtitle file. The subtitle text alone (the meaning) does not say when it appears on screen. The timestamp (the position) is a separate piece of information, attached to every single line. The player needs it to show things in the right order.

> **WAIT — REALLY?**
>
> Attention itself has no built-in sense of order — it compares token pairs regardless of where they sit in the sequence. Without a positional encoding added in, "the model" and "model the" would look completely identical to it. Order is not free. It has to be engineered in, on purpose.

> **KEY TAKEAWAY**
>
> *Each token's final input vector is its meaning (from the embedding table) plus its position (from positional encoding), added together. Meaning without position cannot tell word order apart; this sum fixes that.*

5 The language modelling head: from understanding to one word

One more layer turns the Transformer stack's reasoning into an actual probability for every word in the vocabulary.

Every token is first enriched by the Transformer stack's attention and feed-forward layers, reviewed from Session 3 onward. After that, one more learned layer takes over. This is the **language modelling (LM) head**. It projects the final hidden state into a raw score for every single word in the vocabulary. That means tens of thousands of numbers. The **softmax** function turns those raw scores into probabilities that sum to exactly 1.

WORKED EXAMPLE · *softmax on five candidate next words, by hand*

Say the LM head produces raw scores for one prompt: "The dog movie was really ___". The five candidate scores are great $= 1.80$, good $= 1.35$, slow $= 0.60$, long $= 0.35$, and boring $= -0.30$.

Step 1 **Exponentiate every score.** $e^{1.80} \approx 6.05$, $e^{1.35} \approx 3.86$, $e^{0.60} \approx 1.82$, $e^{0.35} \approx 1.42$, $e^{-0.30} \approx 0.74$.

Step 2 **Sum the exponentials.** $6.05 + 3.86 + 1.82 + 1.42 + 0.74 \approx 13.89$.

Step 3 **Divide each by the sum.** $P(\text{great}) \approx 6.05/13.89 \approx 0.44$, $P(\text{good}) \approx 0.28$, $P(\text{slow}) \approx 0.13$, $P(\text{long}) \approx 0.10$, $P(\text{boring}) \approx 0.05$. These five add up to 1.00.

"Great" is the single most likely next word here, but it is not certain: nearly one prediction in three favours "good" instead.

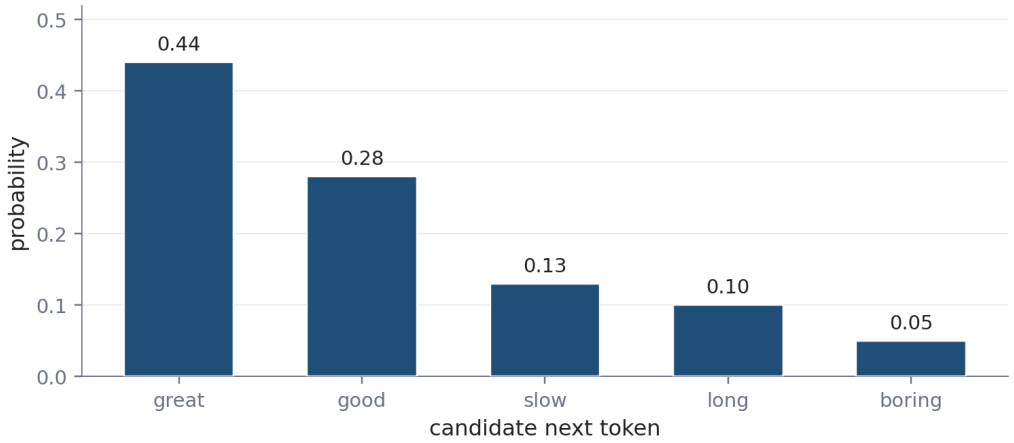


Figure 5. The worked softmax result: a full probability distribution over candidate next words, not a single hard answer.

A **decoding strategy** then has to pick one actual word from this distribution. Always taking the top probability (**greedy decoding**) is one option, among several covered properly in Session 5. Whichever word is chosen gets appended to the sequence. The entire five-stage pipeline from Section 1 runs again to pick the *next* word after that. This repeat-one-token-at-a-time loop is **autoregressive generation**. It is the reason an LLM visibly "types" its answer rather than producing the whole reply instantly.

WATCH OUT

Always picking the single highest-probability word (greedy decoding) can trap a model in dull, repetitive text. The single best next word is sometimes a safe, bland choice, repeated over and over. This is exactly why Session 5 exists: sampling strategies trade some of that safety for variety.

KEY TAKEAWAY

The LM head turns the Transformer's final understanding into a probability over the entire vocabulary, via softmax. A decoding strategy picks one word from that distribution. The loop appends it, and the five-stage pipeline runs again — one token per pass, from the first word to the last.

Where it's used. Every chat assistant you have used — a customer-support bot, a coding assistant, anything — is running this exact five-stage loop. It happens underneath its product polish: tokenize, embed and place, reason through the Transformer stack, score every candidate word, pick one, repeat.

Self-test

1. Why does a 4-gram language model fail to use a relevant word that appeared six words earlier in the sentence?

A 4-gram model only ever conditions its prediction on the previous 3 words ($N-1$ with $N=4$). Anything further back is structurally invisible to it, regardless of how relevant it is.

2. In the BPE worked example, why was `(e, r)` merged before `(l, o)`? "Low" is the most frequent whole word, more frequent than "newer".

BPE counts adjacent *symbol pairs* across the whole weighted corpus, not whole-word frequency. `(e, r)` totalled 11 occurrences, from lower, newer, and wider combined. That beat 9 for `(l, o)`, so it won the first merge despite "low" alone being the single most frequent word.

3. Two tokens end up with nearly identical embedding vectors. What does that actually mean, mechanically?

It means the model's training pushed them to behave similarly for the purpose of predicting surrounding text well. That is not the same as a human labelling them synonyms. Similar usage in context, not shared dictionary meaning, is what the vectors capture.

4. "The dog bit the man" and "the man bit the dog" use identical tokens and identical token embeddings. What makes the model treat them differently, and why is that necessary?

Positional encoding, added on top of each token's embedding. Attention alone compares token pairs with no sense of order. Without a positional vector, these two sentences would look identical to it.

5. The LM head outputs a full probability distribution over the vocabulary, not a single word. What has to happen next, and why might always picking the top probability be a bad idea?

A decoding strategy must pick one word from that distribution. Always picking the single highest-probability word (greedy decoding) can produce dull, repetitive text. The safest word is sometimes chosen over and over.

Pocket card: Session 1 in one page

KEY TAKEAWAY

The whole session, compressed.

- *A language model's entire job is estimating $P(w_t \mid w_1, \ldots, w_{t-1})$ — n-grams count it, neural and Transformer models learn a representation instead.*
- ***Tokenization (BPE)** turns text into reusable sub-word pieces by repeatedly merging the most frequent adjacent pair. This handles rare and unseen words gracefully.*
- ***Embeddings** give each token a meaning vector, learned from context, not a dictionary. **Positional encoding**, added on top, gives it a place in the sentence. Both are necessary; neither alone is enough.*
- *The **LM head** turns the Transformer's final understanding into a probability over the whole vocabulary via softmax. A **decoding strategy** then picks one word.*
- ***Autoregressive generation**: append the chosen word, run the whole five-stage pipeline again, one token per pass.*

Glossary & symbols

Key terms.

language model	a system that assigns a probability to what word comes next, given the words said so far.
n-gram model	a language model that estimates that probability by counting how often a word follows the previous fixed number of words.
token	the smallest unit of text a model actually reads — usually a sub-word piece, not a whole word.
BPE	Byte Pair Encoding; builds a tokenizer's vocabulary by repeatedly merging the most frequent adjacent symbol pair.
embedding	a learned vector meant to capture a token's meaning, based on how it is used in context.
positional encoding	a vector added to each token's embedding that records its position in the sequence.
Transformer stack	the layers of attention and feed-forward computation that process the embedded, positioned sequence.
LM head	the final learned layer that turns the Transformer's output into a raw score for every word in the vocabulary.
softmax	the function that turns a set of raw scores into probabilities that sum to 1.
decoding strategy	the rule used to pick one actual word from the LM head's probability

distribution (e.g. greedy, sampling).

autoregressive generation
generating one token per pass, each new token appended before the next pass runs.

Notation at a glance.

Shorthand	Meaning
w_t	the token (word piece) at position t in the sequence
$P(w_t \mid w_1, \ldots, w_{t-1})$	probability of the next token, given every token before it